

BSDA2001P: Introduction to DL and GenAI Project

Smart MCQ Solver Challenge

Project Report

Detail	Value
Roll Number	23F2003665
Batch	T22026
GitHub Repository	dl-genai-project-26-t2
W&B Project	23f2003665-t22026

1. Abstract / Executive Summary

The Smart MCQ Solver Challenge requires ranking five answer options (A-E) and submitting the top three for each question. The competition metric is MAP@3. I first analysed the dataset and calculated two simple reference baselines: always predicting the three most frequent answer letters (B, C, A) gave 0.4213 MAP@3, while a random top-three baseline gave 0.3675. I then implemented three different models: TF-IDF + Logistic Regression, a TextCNN built from scratch in PyTorch, and a pretrained DistilBERT model from Hugging Face. Their Kaggle MAP@3 scores were 0.75103, 0.74522 and 0.75561 respectively. DistilBERT gave the best score among these models. EDA also showed many duplicate prompts and large train-test prompt overlap, which helps explain why local validation scores can be much higher than Kaggle performance.

2. Introduction

For every MCQ, the model must rank the five options and return the best three in order. My aim was to compare different ways of representing and scoring text: sparse TF-IDF features, a CNN trained from scratch, and a pretrained Transformer. The report follows the final presentation notebook and explains the complete flow of each model from input preparation to final Kaggle submission.

3. Dataset & Preprocessing

Item	Final Notebook Finding
Training set	2,000 questions, 8 columns
Test set	500 questions, 7 columns
Missing values	None in train or test
Duplicate prompt rows in train	242
Unique train prompts	1,758
Unique test prompts	492
Overlapping unique prompts	267 (54.3% of unique test prompts)
Prompt length	Mean 117.67, median 111, max 337 characters

3.1 Exploratory Data Analysis

The answer distribution is mildly imbalanced. B appears most often as the correct answer (24.50%), followed by C (22.95%), A (18.45%), D (17.90%) and E (16.20%). Prompt length was measured in characters. The most important EDA finding was repeated content: 242 training rows repeat a prompt, and 267 unique prompts are shared between train and test after simple lowercase/strip normalization.

3.2 Reference Baselines

Baseline	Method	MAP@3
Most-frequent top-3	Always predict B, C, A	0.4213
Random top-3	Randomly choose three options, seed = 42	0.3675

These two calculations are used only as simple reference baselines. They show what can be achieved without learning any relation between the prompt and answer options.

3.3 General Preprocessing

- Basic text cleaning removes unwanted punctuation and normalizes text where required by the model.
- TF-IDF + Logistic Regression and DistilBERT use prompt-option pairs.
- TextCNN uses one combined sequence containing the prompt and all five options.
- No synthetic data augmentation was used.
- The split is performed before pair expansion so the five options of one original row stay together. Duplicate prompt text under different rows may still cross the split.

4. Tokenization Strategy

The models use different text representations:

TF-IDF does not use neural token embeddings; it converts cleaned text into sparse n-gram features.

TextCNN uses a custom word-level vocabulary with <PAD> and <UNK> tokens and learns its own embeddings from scratch.

DistilBERT uses the pretrained tokenizer associated with distilbert-base-uncased, which tokenizes prompt-option pairs into subword IDs and creates an attention mask.

For both neural models, the final maximum input length used in the notebook is 160.

5. Modeling & Experimentation

5.1 TF-IDF + Logistic Regression

5.1.1 End-to-End Flow

Flow: Clean train/test text -> split original questions 80/20 -> create 5 prompt-option pairs per question -> TF-IDF vectorization -> Logistic Regression -> probability of class 1 for every option -> group 5 option scores by question -> sort scores -> select top 3 -> MAP@3 / submission.

First, the prompt and all five option columns are lowercased, punctuation is removed and repeated spaces are normalized. The original questions are then split into training and validation sets before pair creation. Each question is expanded into five rows. For example, (prompt, A) is one row, (prompt, B) is another, and so on. The pair containing the correct answer receives label 1 and the four incorrect pairs receive label 0.

5.1.2 TF-IDF Representation

For each pair, the cleaned prompt and option are joined into one text string. TfidfVectorizer converts this text into a sparse numerical vector. The final notebook uses unigrams, bigrams and trigrams (`ngram_range=(1,3)`), `sublinear_tf=True`, and at most 50,000 features. The vectorizer is fitted only on training pairs and the same learned vocabulary/IDF mapping is then used to transform validation and test pairs.

5.1.3 Logistic Regression Scoring

Logistic Regression learns a weight for each TF-IDF feature. Conceptually it first computes a linear score $z = w^T x + b$ and then applies a sigmoid to obtain a value between 0 and 1. In this project, `predict_proba(...)[:, 1]` is used because class 1 represents the correct option. Since each question has five pair probabilities, the five options are sorted by this probability and the top three are selected.

Parameter	Value
Train/validation split	80/20, random_state=42
TF-IDF n-grams	(1, 3)
Sublinear TF	True
Maximum features	50,000
LR max_iter	5,000
Class weighting	balanced
LR random_state	42
Validation MAP@3	0.9938
Kaggle MAP@3	0.75103

5.1.4 Validation and Inference

For validation, the pair probabilities are grouped by question ID and ranked. MAP@3 is then calculated using the known validation answers. For test inference, the same pair-building and TF-IDF transformation steps are used, but no labels are available. The top-three option letters are joined as strings such as C A D and written into the sample submission. The very high local validation score compared with Kaggle performance is treated carefully because duplicate prompt content can cross a normal random split.

5.2 TextCNN - Custom Deep Learning Model

5.2.1 End-to-End Flow

Flow: Clean text -> build word vocabulary -> concatenate prompt + A + B + C + D + E -> convert words to token IDs -> pad/truncate to 160 -> stratified train/validation split -> DataLoader batches -> Embedding -> Conv1D -> ReLU -> Global Max Pool -> Linear(5) -> CrossEntropyLoss -> backpropagation with Adam -> validation MAP@3 -> best checkpoint -> test top-3 submission.

5.2.2 Input Preparation

The TextCNN does not create five prompt-option pairs. Instead, the prompt and all five options are concatenated into one sequence. Text is lowercased and non-alphanumeric characters are removed. A custom vocabulary is built from the training text. <PAD> is index 0 and <UNK> is index 1. Every word is replaced by its vocabulary ID, unknown words use <UNK>, shorter sequences are padded, and longer sequences are truncated to 160 word tokens.

The answer letters are converted to class indices: A->0, B->1, C->2, D->3 and E->4. A stratified 85/15 train-validation split keeps the class distribution similar across both parts. Training data is passed through a DataLoader using mini-batches of 32 and shuffling.

5.2.3 Architecture

Layer / Step	Configuration	What it does
Embedding	vocab_size x 64, padding_idx=0	Maps each word ID to a learned 64-dimensional vector.
Permute	(batch, seq, 64) -> (batch, 64, seq)	Changes tensor order because Conv1D expects channels before sequence length.

Conv1D	64 input channels, 64 filters, kernel=3	Slides over three-token windows and learns useful local phrase patterns.
ReLU	$\max(0, x)$	Adds non-linearity by keeping positive activations.
Global Max Pool	Maximum over sequence dimension	Keeps the strongest activation found by each of the 64 filters.
Linear	64 -> 5	Produces five raw logits, one for A, B, C, D and E.

The embedding layer learns word representations, convolution looks for short local patterns, pooling compresses the sequence to 64 features, and the final linear layer maps those features to five answer scores.

5.2.4 Training

The model uses CrossEntropyLoss because exactly one of the five answer classes is correct for each question. For each mini-batch, gradients are cleared, the forward pass produces five logits, CrossEntropyLoss compares those logits with the correct class index, `loss.backward()` calculates gradients, and Adam updates the trainable weights. This repeats for 10 epochs.

Parameter	Final TextCNN Value
Maximum sequence length	160
Batch size	32
Embedding dimension	64
Number of filters	64
Kernel size	3
Loss	CrossEntropyLoss
Optimizer	Adam
Learning rate	1e-3
Epochs	10
Kaggle MAP@3	0.74522

5.2.5 Validation, Checkpoint and Test Prediction

After each epoch, the model is switched to evaluation mode and validation logits are generated without gradient calculation. Softmax is used to convert the five logits to class probabilities for easier interpretation, and the top three class indices are used for MAP@3. When validation MAP@3 improves, the model `state_dict` is saved as the best checkpoint. During test inference, that checkpoint is loaded, the 500 test questions are encoded in the same way, the top-three indices are converted back to A-E, and the submission file is written.

5.3 Fine-Tuned Transformer - DistilBERT

5.3.1 End-to-End Flow

Flow: Clean repeated instruction phrases from prompts -> split original rows 90/10 -> create 5 prompt-option pairs -> DistilBERT tokenizer -> input_ids + attention_mask, max length 160 -> pretrained DistilBERT encoder -> one scalar score per pair -> MSE loss -> Hugging Face Trainer fine-tuning -> validation MAP@3 -> load best saved model -> reshape test scores into groups of 5 -> rank -> submission.

5.3.2 Pair Construction and Tokenization

A separate DistilBERT preprocessing step removes a short list of repeated instruction phrases from prompts, such as 'pick the best possible answer' and 'choose the correct answer'. The original rows are then split 90/10. Each question becomes five (prompt, option) pairs. Correct pairs receive target 1.0 and incorrect pairs receive 0.0. Test pairs do not contain labels.

The pretrained tokenizer for distilbert-base-uncased processes the prompt and option together. It creates input_ids and an attention_mask. Inputs are padded or truncated to 160 tokenizer positions. Hugging Face Dataset.from_list(...).map(..., batched=True) is used to tokenize the pair lists efficiently, and the required columns are returned as PyTorch tensors.

5.3.3 DistilBERT Architecture Used

distilbert-base-uncased is an encoder-only Transformer. The base model uses 6 Transformer encoder blocks, a hidden size of 768 and 12 self-attention heads. In each block, multi-head self-attention lets every token build a contextual representation using the other allowed tokens in the prompt-option sequence. Residual connections and Layer Normalization stabilize the representation, and a feed-forward network performs an additional non-linear transformation for each token.

The pair begins with the model's special first token and contains a separator between the prompt and option. After the Transformer layers, the sequence-classification head uses the final first-token representation to produce the task output. In the notebook, num_labels=1 is used, so the final output is one scalar score for the complete prompt-option pair.

5.3.4 Regression Objective and Fine-Tuning

The model is loaded with problem_type='regression'. Therefore the scalar output is trained against the numeric target 0.0 or 1.0 using Mean Squared Error (MSE). This produces a continuous relevance score for each option. All five scores for a question can then be ranked.

Training is handled by Hugging Face Trainer. No custom optimizer is created in the notebook; optimization is handled by the Trainer configuration. Evaluation and checkpoint saving happen after every epoch. `load_best_model_at_end=True` reloads the checkpoint with the highest validation MAP@3. No layers are explicitly frozen, and no model quantization is used.

Parameter	Value
Model	distilbert-base-uncased
Train/validation split	90/10, random_state=42
Maximum length	160 tokenizer positions
Batch size	16
Epochs	10
Learning rate	2e-5
num_labels	1
Problem type	Regression
Loss	MSE
Evaluation strategy	Every epoch
Save strategy	Every epoch
Best checkpoint metric	Validation MAP@3
Explicitly frozen layers	None
Quantization	Not used
Kaggle MAP@3	0.75561

5.3.5 Validation and Test Inference

DistilBERT predicts one score for each pair, so validation logits are reshaped to (-1, 5). Because the pairs were created in A-E order, each row after reshaping contains the five scores for one original question. These scores are sorted and the top-three indices are compared with the correct answer to calculate MAP@3.

For final inference, the saved fine-tuned DistilBERT model and tokenizer are loaded. `Trainer.predict()` returns one score for every test pair. The predictions are again reshaped into groups of five, the three largest scores are converted to A-E letters, and a fresh copy of the competition sample submission is filled and saved.

6. Performance & Comparative Analysis

6.1 Evaluation Metric - MAP@3

MAP@3 rewards both correctness and ranking position. If the correct answer is at rank 1, the question receives 1.0; at rank 2 it receives 0.5; at rank 3 it receives 1/3; and if the

correct answer is outside the top three, it receives 0. The final MAP@3 is the average across all questions.

6.2 Final Comparison

Type	Method	MAP@3
Reference baseline	Most-frequent B C A	0.4213
Reference baseline	Random top-3	0.3675
Classical ML	TF-IDF + Logistic Regression	0.75103
Custom DL	TextCNN	0.74522
Pretrained model	DistilBERT	0.75561

6.3 Main Findings

- All three trained models clearly outperform the two simple EDA reference baselines.
- DistilBERT gives the best score among the models documented in the final notebook.
- TF-IDF + Logistic Regression is very competitive despite having a much simpler representation and training process.
- TextCNN reaches 0.74522 with a simple one-convolution architecture, 160-token input length and mini-batch training.
- Local validation results must be interpreted carefully because duplicate prompt text can occur across a normal random split.
-

6.4 Training Performance and W&B

Weights & Biases was used to record training information for the neural models. For presentation, the plots used are training loss, evaluation loss and validation MAP@3.

7. Conclusion

This project compared three different ways of solving the same ranking problem.

TF-IDF + Logistic Regression used sparse features and pairwise probabilities.

TextCNN learned word embeddings and local phrase patterns completely from scratch and directly produced five answer logits.

DistilBERT used pretrained contextual representations and pairwise relevance scoring.

The most important evaluation lesson was that duplicate analysis and a reliable validation split are necessary before trusting a very high local score.

7.1 Key Learnings

- How TF-IDF features and Logistic Regression convert text into pairwise correctness probabilities.
- How embeddings, Conv1D, ReLU, global max pooling, CrossEntropyLoss and backpropagation work together in TextCNN.
- How pretrained tokenization, self-attention, Transformer encoders and fine-tuning are used in DistilBERT.
- How MAP@3 changes model evaluation from one hard answer to a ranked top-three prediction.
- How data leakage and duplicate content can create misleading validation scores.